

Parabank Automation Framework

Package Description:

Base:

Contains browser setup, WebDriver initialization, and common reusable methods.

Pages:

Contains Page Object Model (POM) classes with web element locators and page actions.

Tests:

Contains positive test scenarios for ParaBank functionalities.

Negative_Cases:

Contains defect validation and negative test scenarios.

Utils:

Contains utility classes for Excel handling, configuration management, screenshots, reporting, and listeners.

TestData:

Stores Excel-based test data used for data-driven testing.

Reports:

Stores Extent Report generated after test execution.

testNG.xml:

Executes positive test cases.

testNG_Bug.xml:

Executes negative and defect validation test cases.

BaseTest.java:

```
package Base;

import java.time.Duration;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

import org.openqa.selenium.firefox.FirefoxDriver;

import org.testng.ITestResult;

import org.testng.annotations.AfterMethod;

import org.testng.annotations.BeforeMethod;

import org.testng.annotations.BeforeSuite;

import Pages.LoginPage;

import Pages.RegistrationPage;

import Utils.ConfigReader;

import Utils.ScreenshotUtils;

import Utils.UserManager;

public class BaseTest {

    protected WebDriver driver;

    ConfigReader config =

        new ConfigReader();

    @BeforeSuite

    public void createUser() {

        WebDriver tempDriver =

            new ChromeDriver();

        tempDriver.manage()

            .window()

            .maximize();
```

```
tempDriver.get(  
    "https://parabank.parasoft.com/parabank/index.htm");
```

```
RegistrationPage regPage =  
    new RegistrationPage(tempDriver);
```

```
regPage.clickRegisterLink();
```

```
regPage.enterFirstName("Surya");  
regPage.enterLastName("Prakash");  
regPage.enterAddress("Coimbatore");  
regPage.enterCity("Coimbatore");  
regPage.enterState("Tamil Nadu");  
regPage.enterZipCode("641001");  
regPage.enterPhoneNumber("9876543210");  
regPage.enterSSN("123456789");
```

```
String user =  
    "surya"  
    + System.currentTimeMillis();
```

```
UserManager.username = user;
```

```
regPage.enterUsername(user);
```

```
regPage.enterPassword(  
    UserManager.password);
```

```

        regPage.enterConfirmPassword(
            UserManager.password);

        regPage.clickRegisterButton();

        System.out.println(
            "Generated User : "
            + UserManager.username);

        tempDriver.quit();
    }

    @BeforeMethod
    public void setup() {
        String browser =
            config.getBrowser();

        String url =
            config.getUrl();

        if (browser.equalsIgnoreCase("chrome")) {
            driver =
                new ChromeDriver();
        } else if (browser.equalsIgnoreCase("firefox")) {
            driver =
                new FirefoxDriver();
        }

        driver.manage()

```

```
        .window()
        .maximize();
driver.manage()
        .timeouts()
        .implicitlyWait(
            Duration.ofSeconds(10));
driver.get(url);
}
```

```
protected void login() {
    LoginPage loginPage =
        new LoginPage(driver);

    loginPage.enterUsername(
        UserManager.username);

    loginPage.enterPassword(
        UserManager.password);

    loginPage.clickLoginButton();
}
```

@AfterMethod

```
public void tearDown(
    ITestResult result) {
    if (result.getStatus()
        == ITestResult.FAILURE) {
        ScreenshotUtils.captureScreenshot(
```

```

        driver,
        result.getName());
    }
    if (driver != null) {
        driver.quit();
    }
}
}

```

AccountOverviewPage.java:

```

package Pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.Select;

public class AccountOverviewPage {
    WebDriver driver;

    public AccountOverviewPage(WebDriver driver) {
        this.driver = driver;
    }

    By accountsOverviewLink =
        By.linkText("Accounts Overview");

    By accountNumberLink =
        By.xpath(
            "//a[contains(@href,'activity.htm?id=')]");

    By activityPeriodDropdown =
        By.id("month");

    By transactionTypeDropdown =

```

```

        By.id("transactionType");
By goButton =
        By.xpath("//input[@value='Go']");
public void clickAccountsOverviewLink() {
    driver.findElement(
        accountsOverviewLink)
        .click();
}
public boolean isAccountDisplayed() {
    return driver.findElement(
        accountNumberLink)
        .isDisplayed();
}
public void clickAccountNumber() {
    driver.findElement(
        accountNumberLink)
        .click();
}
public void selectActivityPeriod(
    String month) {
    Select select =
        new Select(
            driver.findElement(
                activityPeriodDropdown));
    select.selectByVisibleText(
        month);
}

```

```

public void selectTransactionType(
    String type) {
    Select select =
        new Select(
            driver.findElement(
                transactionTypeDropdown));
    select.selectByVisibleText(
        type);
}

public void clickGoButton() {
    driver.findElement(
        goButton)
        .click();
}
}

```

BillPayPage.java:

```

package Pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.Select;

public class BillPayPage {
    WebDriver driver;

    public BillPayPage(WebDriver driver) {
        this.driver = driver;
    }

    By billPayLink = By.linkText("Bill Pay");
}

```



```

By payeeName = By.name("payee.name");
By address = By.name("payee.address.street");
By city = By.name("payee.address.city");
By state = By.name("payee.address.state");
By zipCode = By.name("payee.address.zipCode");
By phoneNumber = By.name("payee.phoneNumber");
By accountNumber = By.name("payee.accountNumber");
By verifyAccount = By.name("verifyAccount");
By amount = By.name("amount");
By fromAccountDropdown =
    By.name("fromAccountId");

By sendPaymentButton =
    By.cssSelector("input[value='Send Payment']");
public void clickBillPayLink() {
    driver.findElement(billPayLink).click();
}
public void enterPayeeName(String value) {

    driver.findElement(payeeName).sendKeys(value);
}

public void enterAddress(String value) {

    driver.findElement(address).sendKeys(value);
}

```

```
public void enterCity(String value) {

    driver.findElement(city).sendKeys(value);
}

public void enterState(String value) {

    driver.findElement(state).sendKeys(value);
}

public void enterZipCode(String value) {

    driver.findElement(zipCode).sendKeys(value);
}

public void enterPhoneNumber(String value) {

    driver.findElement(phoneNumber).sendKeys(value);
}

public void enterAccountNumber(String value) {

    driver.findElement(accountNumber).sendKeys(value);
}

public void enterVerifyAccount(String value) {

    driver.findElement(verifyAccount).sendKeys(value);
}
```

```

public void enterAmount(String value) {

    driver.findElement(amount).sendKeys(value);
}

public void selectFromAccount() {

    Select select =
        new Select(driver.findElement(fromAccountDropdown));

    select.selectByIndex(0);
}

public void clickSendPaymentButton() {

    driver.findElement(sendPaymentButton).click();
}

public boolean isBillPaymentSuccessful() {

    return driver.getPageSource()
        .contains("was successful");
}
}

```

LoginPage.java:

```

package Pages;

```

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
public class LoginPage {
    WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    By username = By.name("username");

    By password = By.name("password");

    By loginButton =
        By.cssSelector("input[value='Log In']");

    public void enterUsername(String uname) {

        driver.findElement(username).sendKeys(uname);
    }

    public void enterPassword(String pwd) {

        driver.findElement(password).sendKeys(pwd);
    }

    public void clickLoginButton() {
```

```
        driver.findElement(loginButton).click();
    }
}
```

OpenAccountPage.java

```
package Pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.Select;

public class OpenAccountPage {
    WebDriver driver;

    public OpenAccountPage(WebDriver driver) {
        this.driver = driver;
    }

    By openNewAccountLink = By.linkText("Open New Account");

    By accountTypeDropdown = By.id("type");

    By fromAccountDropdown = By.id("fromAccountId");

    By openNewAccountButton =
        By.cssSelector("input[value='Open New Account']");

    public void clickOpenNewAccountLink() {

        driver.findElement(openNewAccountLink).click();
    }
}
```

```
}
```

```
public void selectAccountType(String accountType) {
```

```
    Select select =
```

```
        new Select(driver.findElement(accountTypeDropdown));
```

```
    select.selectByVisibleText(accountType);
```

```
}
```

```
public void selectExistingAccount() {
```

```
    Select select =
```

```
        new Select(driver.findElement(fromAccountDropdown));
```

```
    select.selectByIndex(0);
```

```
}
```

```
public void clickOpenNewAccountButton() {
```

```
    driver.findElement(openNewAccountButton).click();
```

```
}
```

```
}
```

Registration.java:

```
package Pages;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
public class RegistrationPage {

    WebDriver driver;

    public RegistrationPage(WebDriver driver) {
        this.driver = driver;
    }

    By registerLink = By.xpath("//*[@id='loginPanel']/p[2]/a");

    By firstName = By.id("customer.firstName");
    By lastName = By.id("customer.lastName");
    By address = By.id("customer.address.street");
    By city = By.id("customer.address.city");
    By state = By.id("customer.address.state");
    By zipCode = By.id("customer.address.zipCode");
    By phoneNumber = By.id("customer.phoneNumber");
    By ssn = By.id("customer.ssn");
    By username = By.id("customer.username");
    By password = By.id("customer.password");
    By confirmPassword = By.id("repeatedPassword");

    By registerButton = By.cssSelector("input[value='Register']");
    public void clickRegisterLink() {
        driver.findElement(registerLink).click();
    }
}
```

```
public void enterFirstName(String fname) {  
    driver.findElement(firstName).sendKeys(fname);  
}
```

```
public void enterLastName(String lname) {  
    driver.findElement(lastName).sendKeys(lname);  
}
```

```
public void enterAddress(String addr) {  
    driver.findElement(address).sendKeys(addr);  
}
```

```
public void enterCity(String cityName) {  
    driver.findElement(city).sendKeys(cityName);  
}
```

```
public void enterState(String stateName) {  
    driver.findElement(state).sendKeys(stateName);  
}
```

```
public void enterZipCode(String zip) {  
    driver.findElement(zipCode).sendKeys(zip);  
}
```

```
public void enterPhoneNumber(String phone) {  
    driver.findElement(phoneNumber).sendKeys(phone);  
}
```



```
}
```

```
public void enterSSN(String ssnNo) {  
    driver.findElement(ssn).sendKeys(ssnNo);  
}
```

```
public void enterUsername(String uname) {  
    driver.findElement(username).sendKeys(uname);  
}
```

```
public void enterPassword(String pwd) {  
    driver.findElement(password).sendKeys(pwd);  
}
```

```
public void enterConfirmPassword(String cpwd) {  
    driver.findElement(confirmPassword).sendKeys(cpwd);  
}
```

```
public void clickRegisterButton() {  
    driver.findElement(registerButton).click();  
}  
}
```

TransferFundsPage.java

```
package Pages;  
  
import java.time.Duration;  
  
import org.openqa.selenium.By;  
  
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;
public class TransferFundsPage {
    WebDriver driver;
    WebDriverWait wait;

    public TransferFundsPage(WebDriver driver) {

        this.driver = driver;
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    By transferFundsLink = By.linkText("Transfer Funds");

    By amount = By.id("amount");

    By fromAccountDropdown = By.id("fromAccountId");

    By toAccountDropdown = By.id("toAccountId");

    By transferButton =
        By.cssSelector("input[value='Transfer']");

    By transferCompleteMessage =
        By.xpath("//h1[text()='Transfer Complete!']");
```

```
public void clickTransferFundsLink() {

    wait.until(

        ExpectedConditions.elementToBeClickable(

            transferFundsLink));

    driver.findElement(transferFundsLink).click();
}

public void enterAmount(String transferAmount) {

    wait.until(

        ExpectedConditions.visibilityOfElementLocated(

            amount));

    driver.findElement(amount).clear();

    driver.findElement(amount).sendKeys(transferAmount);
}

public void selectFromAccount() {

    Select select =

        new Select(driver.findElement(fromAccountDropdown));

    select.selectByIndex(0);
}
```

```
public void selectToAccount() {

    Select select =

        new Select(driver.findElement(toAccountDropdown));

    if (select.getOptions().size() > 1) {

        select.selectByIndex(1);

    } else {

        select.selectByIndex(0);

    }
}

public void selectSameAccount() {

    Select fromAccount =

        new Select(driver.findElement(

            By.id("fromAccountId")));

    String account =

        fromAccount.getFirstSelectedOption()

            .getText();

    Select toAccount =

        new Select(driver.findElement(

            By.id("toAccountId")));
```

```

        toAccount.selectByVisibleText(account);
    }

    public void clickTransferButton() {

        wait.until(
            ExpectedConditions.elementToBeClickable(
                transferButton));

        driver.findElement(transferButton).click();
    }

    public boolean isTransferSuccessful() {

        wait.until(
            ExpectedConditions.visibilityOfElementLocated(
                transferCompleteMessage));

        return driver.findElement(
            transferCompleteMessage).isDisplayed();
    }
}

```

AccountOverviewTest.java

```

package Tests;

import org.testng.Assert;

import org.testng.annotations.Test;

```

```
import Base.BaseTest;

import Pages.AccountOverviewPage;

public class AccountOverviewTest

    extends BaseTest {

    @Test(priority = 1)

    public void accountOverviewTest() {

        login();

        AccountOverviewPage accountPage =

            new AccountOverviewPage(

                driver);

        accountPage

            .clickAccountsOverviewLink();

        Assert.assertTrue(

            accountPage

                .isAccountDisplayed(),

            "Account details not displayed");

    }

    @Test(priority = 2)

    public void accountActivityFilterTest() {

        login();
```

```
AccountOverviewPage accountPage =  
    new AccountOverviewPage(  
        driver);  
  
accountPage  
    .clickAccountsOverviewLink();  
  
accountPage  
    .clickAccountNumber();  
  
accountPage  
    .selectActivityPeriod(  
        "All");  
  
accountPage  
    .selectTransactionType(  
        "All");  
  
accountPage  
    .clickGoButton();  
  
Assert.assertTrue(  
    driver.getPageSource()  
        .contains(  
            "Account Activity"),  
    "Account Activity page not displayed");  
}
```

```
}
```

BillpayTest.java

```
package Tests;

import org.testng.Assert;
import org.testng.annotations.Test;
import Base.BaseTest;
import Pages.BillPayPage;
import Utils.ExcelUtils;

public class BillPayTest extends BaseTest {

    @Test(priority = 1)
    public void billPayTest() {
        login();

        String payeeName =
            ExcelUtils.getCellData("BillPay", 1, 0);

        String address =
            ExcelUtils.getCellData("BillPay", 1, 1);

        String city =
            ExcelUtils.getCellData("BillPay", 1, 2);

        String state =
            ExcelUtils.getCellData("BillPay", 1, 3);

        String zipCode =
            ExcelUtils.getCellData("BillPay", 1, 4);

        String phoneNumber =
```



```
ExcelUtils.getCellData("BillPay", 1, 5);
```

```
String accountNumber =
```

```
ExcelUtils.getCellData("BillPay", 1, 6);
```

```
String amount =
```

```
ExcelUtils.getCellData("BillPay", 1, 7);
```

```
BillPayPage billPayPage =
```

```
new BillPayPage(driver);
```

```
billPayPage.clickBillPayLink();
```

```
billPayPage.enterPayeeName(payeeName);
```

```
billPayPage.enterAddress(address);
```

```
billPayPage.enterCity(city);
```

```
billPayPage.enterState(state);
```

```
billPayPage.enterZipCode(zipCode);
```

```
billPayPage.enterPhoneNumber(phoneNumber);
```

```
billPayPage.enterAccountNumber(accountNumber);
```

```
billPayPage.enterVerifyAccount(accountNumber);
```

```
billPayPage.enterAmount(amount);
```

```
billPayPage.selectFromAccount();
```

```
billPayPage.clickSendPaymentButton();
```

```
Assert.assertTrue(  
    billPayPage.isBillPaymentSuccessful(),  
    "Bill Payment Failed");  
}
```

```
}
```

OpenAccountTest.java

```
package Tests;  
  
import org.testng.Assert;  
import org.testng.annotations.Test;  
import Base.BaseTest;  
import Pages.OpenAccountPage;  
  
public class OpenAccountTest extends BaseTest {  
  
    @Test  
    public void openAccountTest() {  
        login();  
  
        OpenAccountPage openAccountPage =  
            new OpenAccountPage(driver);  
        openAccountPage.clickOpenNewAccountLink();  
        openAccountPage.selectAccountType("SAVINGS");  
  
        openAccountPage.selectExistingAccount();  
  
        openAccountPage.clickOpenNewAccountButton();  
  
        Assert.assertTrue(  

```

```

        driver.getPageSource()

            .contains("Account Opened!"),

            "Account Creation Failed");

    }

}

```

RegistrationTest.java

```

package Tests;

import org.testng.Assert;

import org.testng.annotations.DataProvider;

import org.testng.annotations.Test;

import Base.BaseTest;

import Pages.RegistrationPage;

import Utils.ExcelUtils;

public class RegistrationTest extends BaseTest {

    private static int outputRow = 1;

    @DataProvider(name = "registrationData")

    public Object[][] getData() {

        int rowCount =

            ExcelUtils.getRowCount(

                "Registration");

        Object[][] data =

            new Object[rowCount][10];

        for (int i = 1; i <= rowCount; i++) {

```

```

        for (int j = 0; j < 10; j++) {

            data[i - 1][j] =

                ExcelUtils.getCellData(

                    "Registration",

                    i,

                    j);

        }

    }

    return data;
}

@Test(dataProvider = "registrationData")
public void registrationTest(

    String firstName,

    String lastName,

    String address,

    String city,

    String state,

    String zipCode,

    String phone,

    String ssn,

    String username,

    String password) {

    RegistrationPage regPage =

```

```
new RegistrationPage(driver);
```

```
regPage.clickRegisterLink();
```

```
regPage.enterFirstName(firstName);
```

```
regPage.enterLastName(lastName);
```

```
regPage.enterAddress(address);
```

```
regPage.enterCity(city);
```

```
regPage.enterState(state);
```

```
regPage.enterZipCode(zipCode);
```

```
regPage.enterPhoneNumber(phone);
```

```
regPage.enterSSN(ssn);
```

```
String uniqueUsername =
```

```
    username +
```

```
        System.currentTimeMillis();
```

```
regPage.enterUsername(
```

```
    uniqueUsername);
```

```
regPage.enterPassword(password);
```

```
regPage.enterConfirmPassword(password);
```

```
regPage.clickRegisterButton();
```

```
ExcelUtils.writeData(  
    "RegisteredUsers",  
    outputRow,  
    0,  
    uniqueUsername);
```

```
ExcelUtils.writeData(  
    "RegisteredUsers",  
    outputRow,  
    1,  
    password);
```

```
ExcelUtils.writeData(  
    "RegisteredUsers",  
    outputRow,  
    2,  
    "PASS");
```

```
outputRow++;
```

```
System.out.println(
```

```
"Generated Username : "  
    + uniqueUsername);
```

```
Assert.assertTrue(  
    driver.getPageSource()  
        .contains("Welcome"),  
    "Registration Failed");  
System.out.println(  
    "Generated Username : "  
    + uniqueUsername);
```

```
Assert.assertTrue(  
    driver.getPageSource()  
        .contains("Welcome"),  
    "Registration Failed");  
}
```

```
}
```

TransferFundsTest.java

```
package Tests;  
  
import org.testng.Assert;  
import org.testng.annotations.Test;  
import Base.BaseTest;  
import Pages.TransferFundsPage;  
  
public class TransferFundsTest extends BaseTest {  
    @Test(priority = 1)  
    public void transferFundsTest() {  
        login();
```

```
TransferFundsPage transferPage =  
    new TransferFundsPage(driver);  
  
transferPage.clickTransferFundsLink();  
  
transferPage.enterAmount("500");  
  
transferPage.selectFromAccount();  
  
transferPage.selectToAccount();  
  
transferPage.clickTransferButton();  
  
Assert.assertTrue(  
    transferPage.isTransferSuccessful(),  
    "Fund Transfer Failed");  
}  
}
```

ConfigReader.java

```
package Utils;  
  
import java.io.FileInputStream;  
import java.util.Properties;  
  
public class ConfigReader {  
    static Properties prop;  
  
    public ConfigReader() {  
        try {  
            FileInputStream fis =
```



```
        new FileInputStream(
            "src/test/resources/config.properties");
    prop = new Properties();
    prop.load(fis);
} catch (Exception e) {
    e.printStackTrace();
}
}
```

```
public String getBrowser() {

    return prop.getProperty("browser");
}
```

```
public String getUrl() {

    return prop.getProperty("url");
}
```

```
public String getUsername() {

    return prop.getProperty("username");
}
```

```
public String getPassword() {

    return prop.getProperty("password");
}
```

```

    }

    public String getExecution() {

        return prop.getProperty("execution");

    }

}

```

ExcelUtils.java

```

package Utils;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import org.apache.poi.ss.usermodel.Cell;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;
import org.apache.poi.ss.usermodel.WorkbookFactory;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class ExcelUtils {

    private static final String filePath =

        "src/test/resources/TestData/TestData.xlsx";

    public static void writeData(

        String sheetName,

        int rowNum,

        int colNum,

        String value) {

```

```
try {

    FileInputStream fis =
        new FileInputStream(
            filePath);

    Workbook workbook =
        WorkbookFactory.create(fis);

    Sheet sheet =
        workbook.getSheet(
            sheetName);

    Row row =
        sheet.getRow(rowNum);

    if(row == null) {

        row =
            sheet.createRow(rowNum);
    }

    Cell cell =
        row.getCell(colNum);

    if(cell == null) {
```

```

        cell =
            row.createCell(colNum);
    }

    cell.setCellValue(value);

    fis.close();

    FileOutputStream fos =
        new FileOutputStream(
            filePath);

    workbook.write(fos);

    fos.close();

    workbook.close();

} catch (Exception e) {

    e.printStackTrace();
}

}

public static String getCellData(
    String sheetName,
    int rowNum,

```

```
int colNum) {

try {

    FileInputStream fis =
        new FileInputStream(
            "src/test/resources/TestData/TestData.xlsx");

    XSSFWorkbook workbook =
        new XSSFWorkbook(fis);

    XSSFSheet sheet =
        workbook.getSheet(sheetName);

    String value =
        sheet.getRow(rowNum)
            .getCell(colNum)
            .toString();

    workbook.close();

    return value;

} catch (Exception e) {

    e.printStackTrace();
```

```

        return "";
    }
}

public static int getRowCount(
    String sheetName) {

    try {

        FileInputStream fis =
            new FileInputStream(
                "src/test/resources/TestData/TestData.xlsx");

        XSSFWorkbook workbook =
            new XSSFWorkbook(fis);

        XSSFSheet sheet =
            workbook.getSheet(sheetName);

        int rows =
            sheet.getLastRowNum();

        workbook.close();

        return rows;

    } catch (Exception e) {

```

```
        e.printStackTrace();

        return 0;
    }
}
}
```

ExtentManager.java

```
package Utils;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;

public class ExtentManager {

    private static ExtentReports extent;

    public static ExtentReports getInstance() {

        if (extent == null) {

            ExtentSparkReporter spark =
                new ExtentSparkReporter(
                    "Reports/ExtentReport.html");

            extent = new ExtentReports();

            extent.attachReporter(spark);
```

```

        extent.setSystemInfo(
            "Tester",
            "Surya Prakash");

        extent.setSystemInfo(
            "Project",
            "ParaBank Automation");
    }

    return extent;
}
}

```

ScreenshotUtils.java

```

package Utils;

import java.io.File;
import java.io.IOException;
import org.apache.commons.io.FileUtils;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

public class ScreenshotUtils {

    public static void captureScreenshot(
        WebDriver driver,
        String testName) {

        File src =

```



```

        ((TakesScreenshot) driver)
            .getScreenshotAs(
                OutputType.FILE);

File dest =
    new File(
        "Screenshots/"
            + testName
            + ".png");

try {

    FileUtils.copyFile(src, dest);

} catch (IOException e) {

    e.printStackTrace();
}
}
}

```

TestListener.java

```

package Utils;

import org.testng.ITestContext;
import org.testng.ITestListener;
import org.testng.ITestResult;
import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;

```

```
public class TestListener
    implements ITestListener {
    ExtentReports extent =
        ExtentManager.getInstance();

    ExtentTest test;

    @Override
    public void onTestStart(
        ITestResult result) {

        test = extent.createTest(
            result.getName());
    }

    @Override
    public void onTestSuccess(
        ITestResult result) {

        test.pass(
            "Test Passed Successfully");
    }

    @Override
    public void onTestFailure(
        ITestResult result) {
```

```
test.fail(  
    "Test Failed");
```

```
test.fail(  
    result.getThrowable());  
}
```

```
@Override  
public void onTestSkipped(  
    ITestResult result) {
```

```
test.skip(  
    "Test Skipped");  
}
```

```
@Override  
public void onFinish(  
    ITestContext context) {  
    extent.flush();  
}  
}
```

config.properties:

browser=chrome

url=https://parabank.parasoft.com/parabank/index.htm



Jenkinsfile

```
pipeline { agent any

tools {
    maven 'Maven installations'
    jdk 'jdk21'
}

stages {

    stage('Checkout') {
        steps {
            git branch: 'main',
                url: 'https://github.com/suryaprakashgithubj/Capstone\_Project.git'
        }
    }
    stage('Build') {
        steps {
            dir('parabank-automation-framework') {
                script {
                    if (isUnix()) {
                        sh 'mvn clean compile'
                    } else {
                        bat 'mvn clean compile'
                    }
                }
            }
        }
    }

    stage('Run Tests') {
        steps {
            dir('parabank-automation-framework') {
                script {
                    if (isUnix()) {
                        sh 'mvn test'
                    } else {
                        bat 'mvn test'
                    }
                }
            }
        }
    }
}
```

```

    }
  }
}

post {
  always {
    archiveArtifacts artifacts: '**/target/surefire-reports/*',
      allowEmptyArchive: true
  }

  success {
    echo 'Build Successful!'
  }

  failure {
    echo 'Build Failed!'
  }
}

```

testNg.xml

```

<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="ParaBank Suite">
  <listeners>
    <listener class-name="Utils.TestListener"/>
  </listeners>

  <test name="ParaBank Tests">
    <classes>
      <class name="Tests.RegistrationTest"/>
      <class name="Tests.LoginTest"/>
      <class name="Tests.OpenAccountTest"/>
      <class name="Tests.TransferFundsTest"/>
      <class name="Tests.BillPayTest"/>
    </classes>
  </test>
</suite>

```

</classes>

</test></suite>

Console Output:

```
History Synchronize Git Staging Git Reflog Properties Console Progress Results of running Chat
<terminated> parbank-automation-framework_testNG.xml [TestNG] /Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java (Jun 11, 202
Generated Username : meena1781171638539
Jun 11, 2026 3:24:00 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 149
Jun 11, 2026 3:24:00 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$4
WARNING: Unable to find version of CDP to use for 149.0.7827.103. You may need to include a dep
Generated Username : sanjay1781171646416
Generated Username : sanjay1781171646416
Jun 11, 2026 3:24:08 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 149
Jun 11, 2026 3:24:08 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$4
WARNING: Unable to find version of CDP to use for 149.0.7827.103. You may need to include a dep
Generated Username : anitha1781171659611
Generated Username : anitha1781171659611
Jun 11, 2026 3:24:22 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 149
Jun 11, 2026 3:24:22 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$4
WARNING: Unable to find version of CDP to use for 149.0.7827.103. You may need to include a dep
Generated Username : vikram1781171669135
Generated Username : vikram1781171669135
Jun 11, 2026 3:24:31 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 149
Jun 11, 2026 3:24:31 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$4
WARNING: Unable to find version of CDP to use for 149.0.7827.103. You may need to include a dep
Jun 11, 2026 3:24:40 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 149
Jun 11, 2026 3:24:40 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$4
WARNING: Unable to find version of CDP to use for 149.0.7827.103. You may need to include a dep
Jun 11, 2026 3:24:46 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 149
Jun 11, 2026 3:24:46 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$4
WARNING: Unable to find version of CDP to use for 149.0.7827.103. You may need to include a dep
Jun 11, 2026 3:24:58 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 149
Jun 11, 2026 3:24:58 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$4
WARNING: Unable to find version of CDP to use for 149.0.7827.103. You may need to include a dep

=====
ParaBank Suite
Total tests run: 14, Passes: 14, Failures: 0, Skips: 0
=====
```